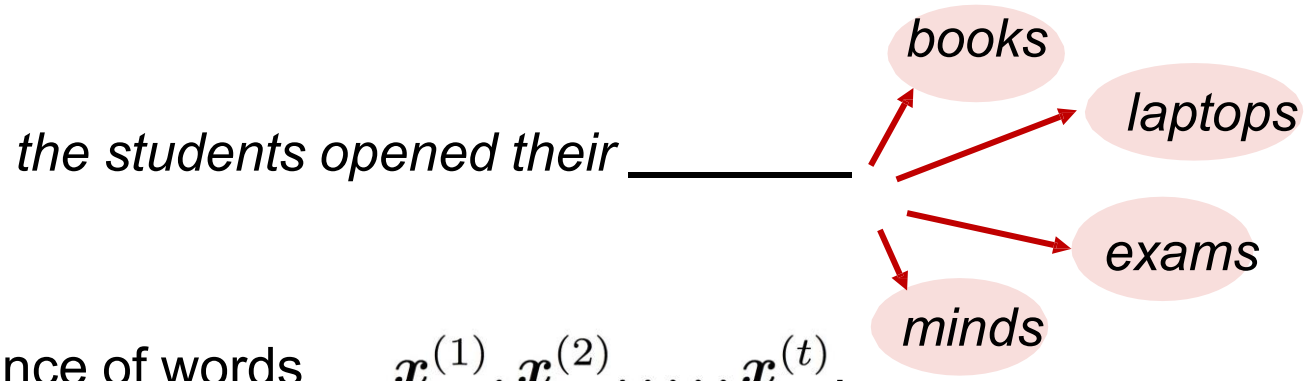
Language Models and Small Language Models

Lecture Overview

- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

Language Modeling

- **Language Modeling** is the task of predicting what word comes next



- More formally: given a sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$, compute the probability distribution of the next word $\boldsymbol{x}^{(t+1)}$:

$$P(\boldsymbol{x}^{(t+1)} | \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$$

where $\boldsymbol{x}^{(t+1)}$ can be any word in the vocabulary $V = \{\boldsymbol{w}_1, \ldots, \boldsymbol{w}_{|V|}\}$

- A system that does this is called a **Language Model**

Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text

- For example, if we have some text $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, then the probability of this text (according to the Language Model) is:

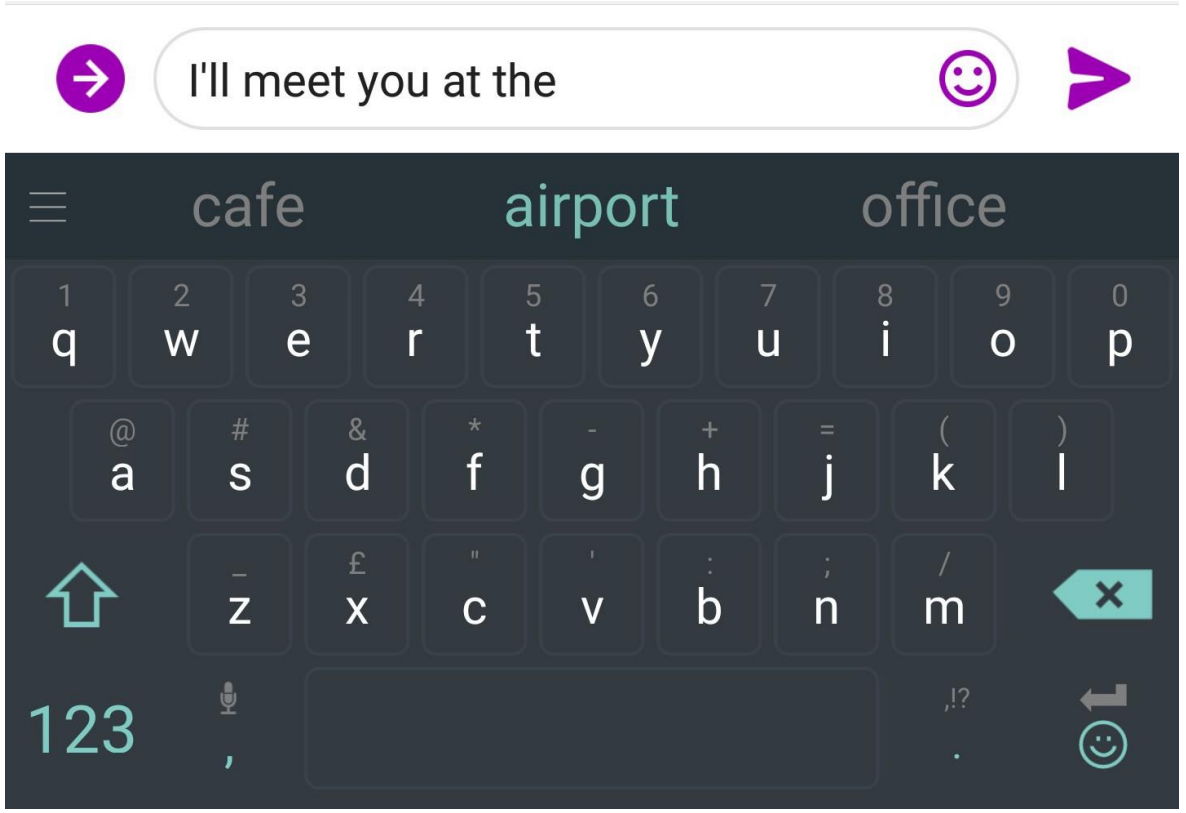
$$P(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = P(\boldsymbol{x}^{(1)}) \times P(\boldsymbol{x}^{(2)} | \boldsymbol{x}^{(1)}) \times \cdots \times P(\boldsymbol{x}^{(T)} | \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)})$$

$$= \prod_{t=1}^{T} \underbrace{P(\boldsymbol{x}^{(t)} | \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)})}$$

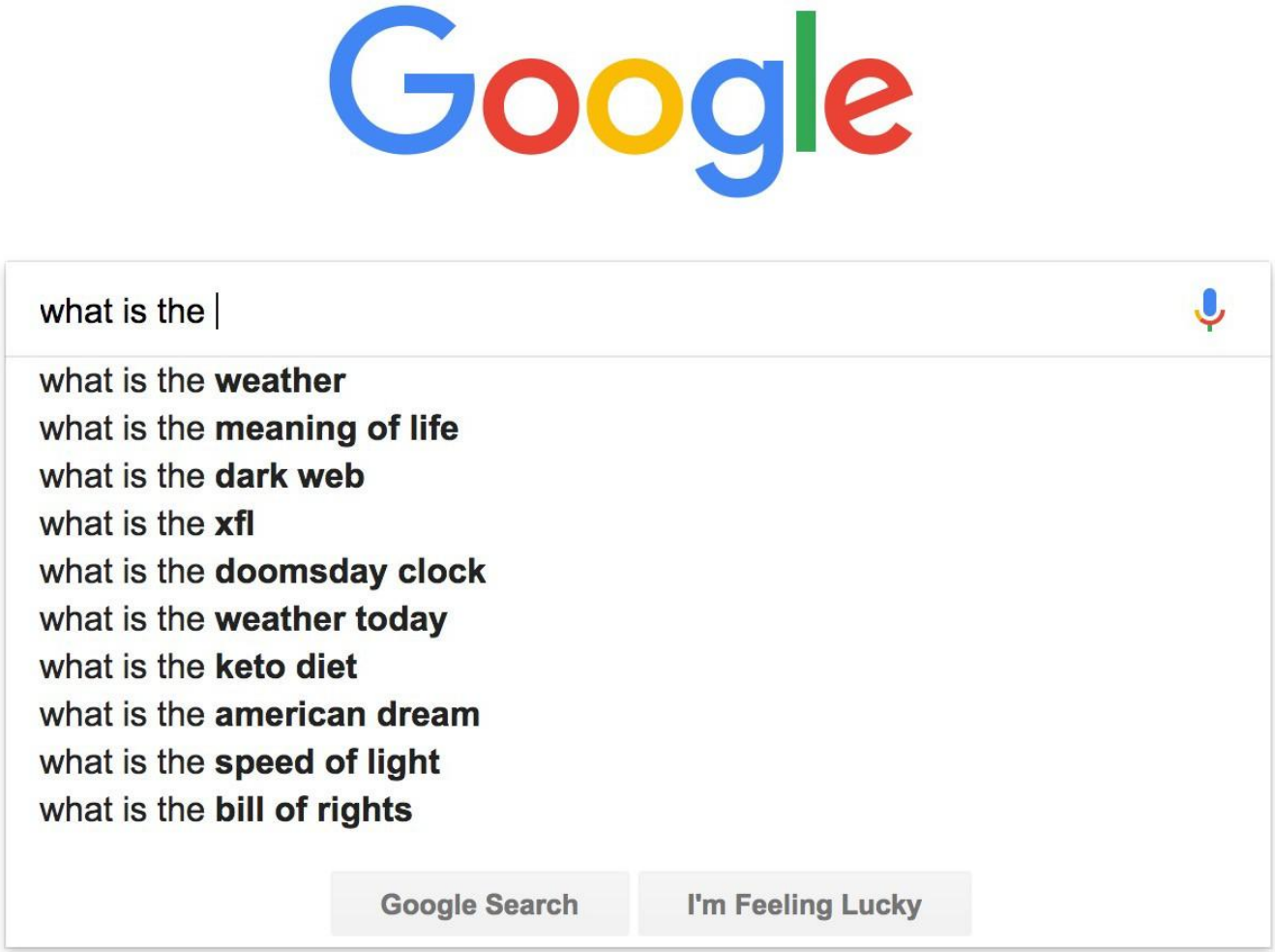
This is what our LM provides

You use Language Models every day!

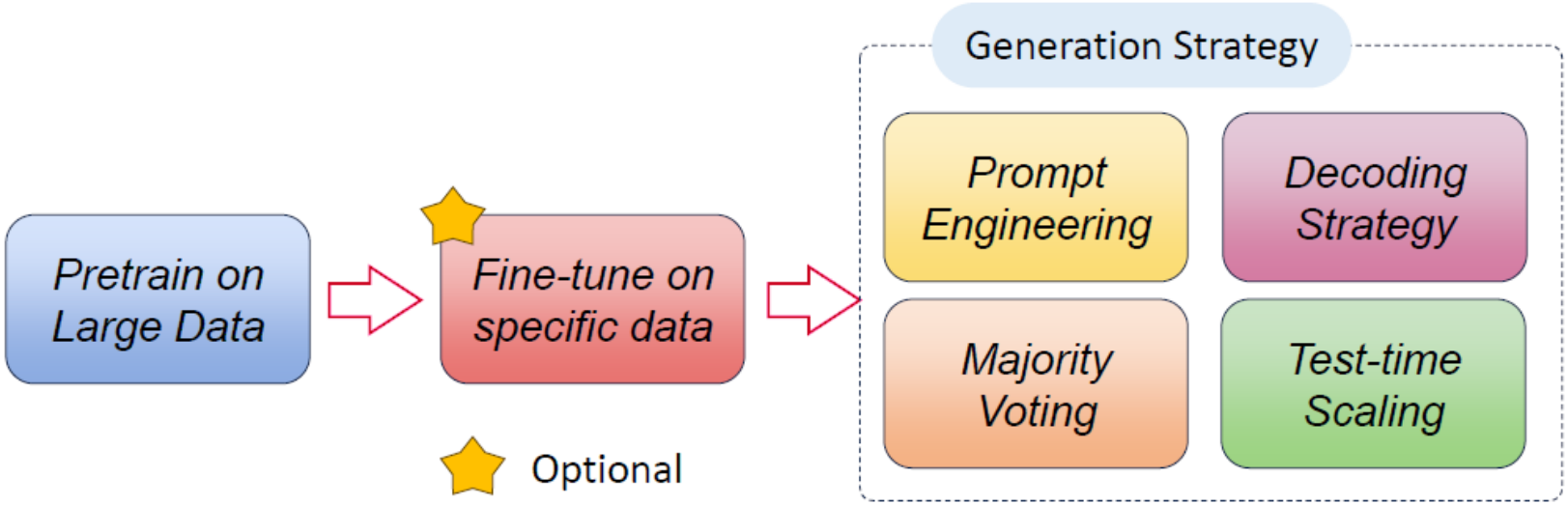
- Language Models are the cornerstone of human language technology
- Everything you do with computers that involves human language uses a language model



You use Language Models every day!



A typical LLM workflow: from pretraining to inference



Pretraining: Transformer, Training Scaling
Fine-tuning: Parameter-efficient fine-tuning, Reinforcement learning
Generation Strategy: Prompt Engineering, Decoding Strategy, Majority Voting, Test-time Scaling

Lecture Overview

- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

You are the engineer

Build an AI tutor for a university.

- Must work on a normal laptop
- Must respond quickly
- Students may share private information
- Internet connectivity is not guaranteed
- No budget for expensive cloud inference on every question

Would you deploy the largest possible language model?

Why Small Language Models?

Large models deliver broad capability at a cost. SLMs trade general-purpose capacity for lower memory, latency, and deployment cost — opening use cases where LLMs are impractical.

On-device deployment

Run locally on phones, IoT, and embedded hardware.

Inference latency

Fewer parameters yield faster, more predictable responses.

Privacy

Local processing keeps sensitive data off the cloud.

Operational cost

Lower compute and energy per query at scale.

Fine-tuning cost

Cheaper to adapt and personalize to a task.

Specialization

Focused models can rival generalists on narrow tasks.

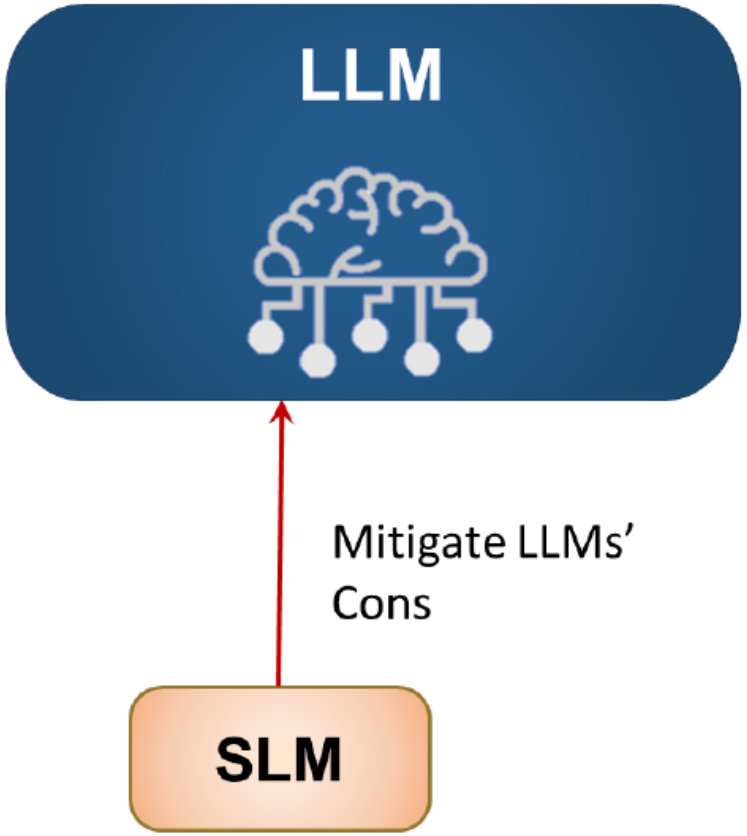
Cloud-scale LLM

- Tens–hundreds of billions of parameters
- Served from datacenters; network dependent
- Highest general capability and reasoning
- High memory, latency, and per-query cost

Resource-efficient SLM

- Millions to a few billion parameters
- Runs on-device; works offline, privately
- Strong on targeted, specialized tasks
- Low memory, latency, and operating cost

Why small language models (SLMs)?



Pros:

- Emergent ability
- Generalizability

Cons:

- Privacy leakage
- On-device deployment
- Inference latency
- Expensive fine-tuning
- Inferior to specialized models

SLMs serve a different set of needs—they open up new possibilities where LLMs fall short.

SLMs and LLMs Address Different Constraints

An SLM is not merely a smaller, weaker LLM. The two occupy different points on a multi-objective trade-off; each dominates the other on some axes.

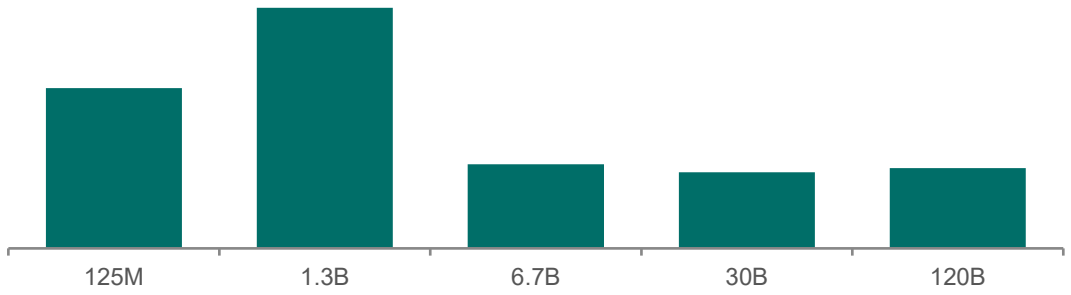
	General capability	Latency	Memory footprint	Privacy (local)	Task specialization
Large Language Model	●●● High	●○○ Higher	●○○ Large	●○○ Weaker	●●○ Broad, not focused
Small Language Model	●●○ Moderate	●●● Lower	●●● Small	●●● Stronger	●●● Can exceed on-task

Model size alone does not define an SLM; task requirements and resource constraints are equally decisive.

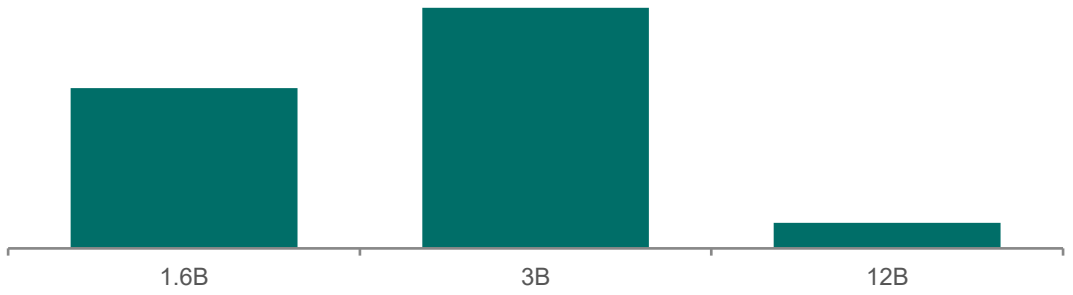
Growth of the SLM Ecosystem

Within model families, the smaller variants are downloaded far more often than the largest ones — evidence of real, growing demand for efficient models.

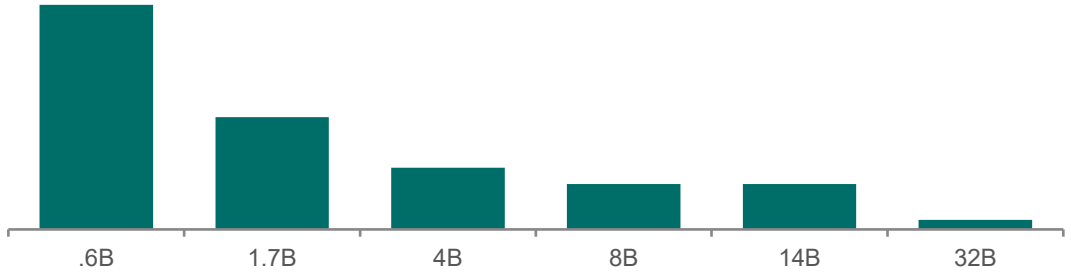
Galactica — downloads by size *values in K*



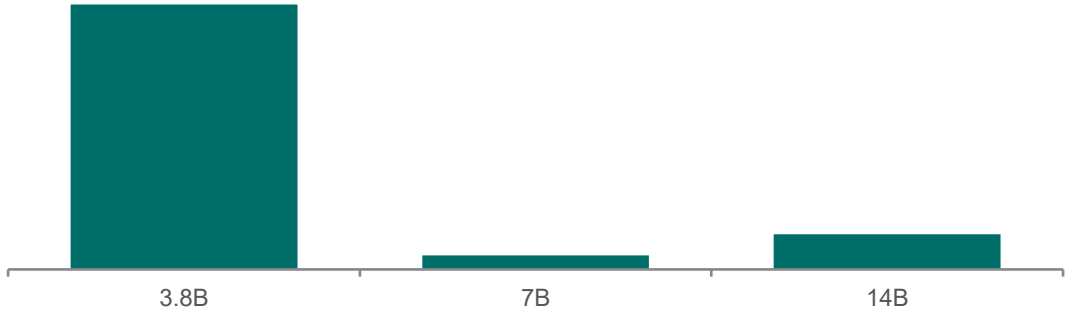
StableLM-2 — downloads by size *values in K*



Qwen-3 — downloads by size *values in M*



Phi-3 — downloads by size *values in K*



Evolution of Small Language Models

A steady, accelerating stream of SLMs from diverse groups — both general-purpose families and domain-specific models.



- MobileBERT
- T5 / Flan-T5
- GPT-Neo
- Megatron-GPT2

- OPT
- Galactica
- BLOOM
- Cerebras-GPT

- Pythia
- Phi-1 / 1.5
- TinyLlama
- Qwen-1, StableLM

- Phi-2 / 3
- Gemma, MiniCPM
- MobileLLM, OpenELM
- OLMo, SmolLM

- Qwen-2.5
- Llama-3.2
- Phi-3.5, PhoneLM
- Fox-1, Rene

Domain-specific SLMs

Healthcare

BioGPT · Meditron · BioMistral

Science

SciGLM · Llemma

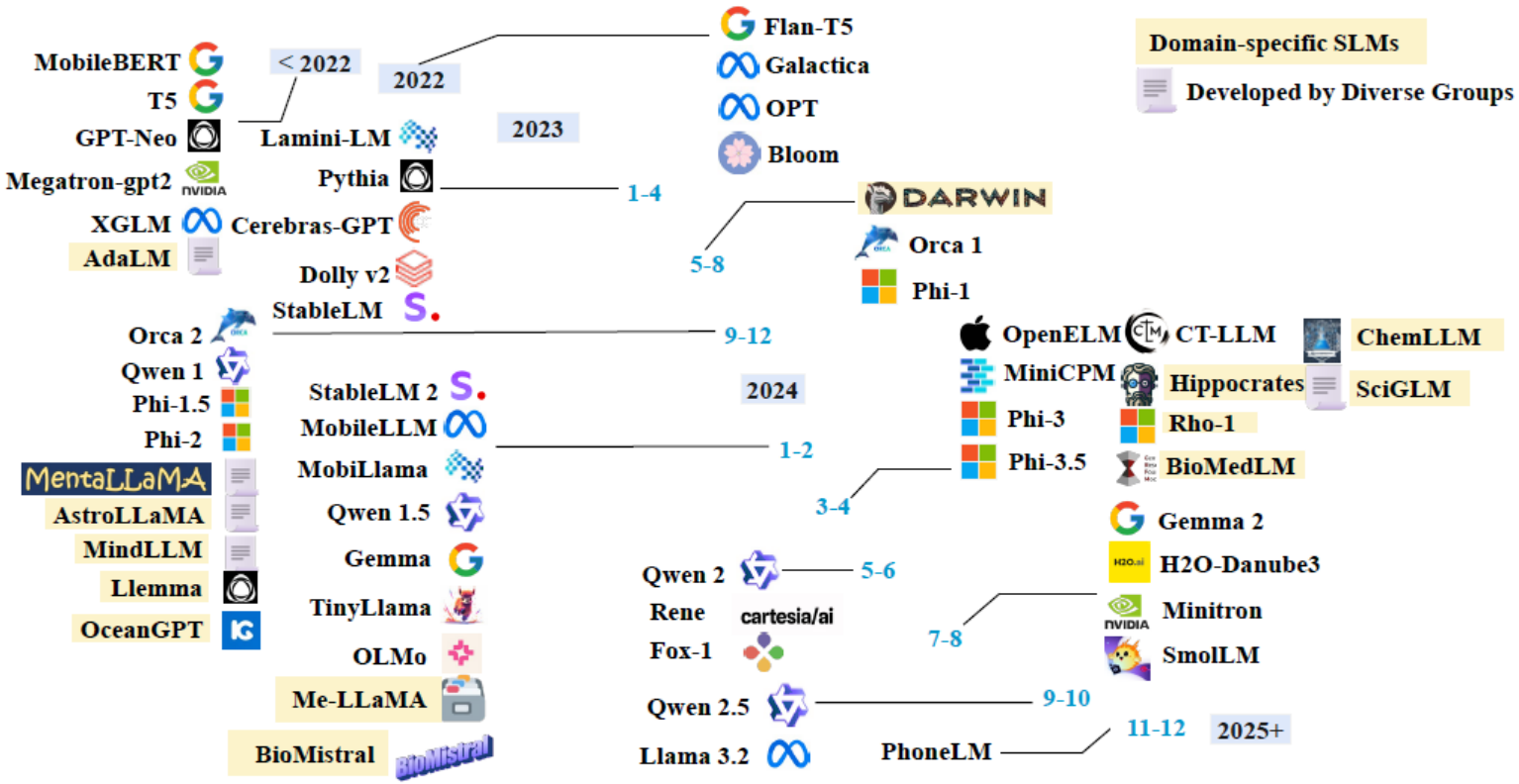
Chemistry

ChemLLM

Ocean / Astro

OceanGPT · AstroLLaMA

Timeline of existing SLMs



Evolution of small language models over time

- There is a steady and accelerating stream of SLMs
- SLMs are actively evolving as a research and engineering frontier.

Lecture Overview

- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

What Counts as a Small Language Model?

There is no agreed size cutoff. Three common views:

Relative definition

“Small” is defined relative to today’s largest models: anything smaller than current frontier LLMs is treated as small. The boundary moves as LLMs grow.

Mobile-device definition

MobileLLM categorizes SLMs as models with fewer than one billion parameters — suited to phones with roughly 6 GB of memory.

Capability / emergence

SLMs span a few million to a few billion parameters (under ~7B–10B), often lacking emergent abilities and needing extra strategies to match LLM reasoning.

Open question: the views disagree and draw no sharp SLM–LLM boundary. Is a 7B model an SLM or an LLM? It depends on the task and the deployment budget, which motivates a task- and resource-aware definition.

A Task- and Resource-Aware Definition

Rather than a fixed cutoff, define an SLM as a range bounded below by capability and above by resources — for a given task and deployment budget.



This lens is flexible: the same model may be an "SLM" for one task/hardware pairing and inadequate for another.

The SLM Design Space

The rest of this lecture is organized around six design areas that recur across the SLM lifecycle.

01

Architecture

Efficient attention, SSMs, xLSTM, routing

02

Training

SFT, PEFT, distillation, synthetic data

03

Specialization

Task adaptation that offsets limited scale

04

Inference

Decoding, KV cache, test-time scaling

05

Deployment

On-device, cloud–edge, compression

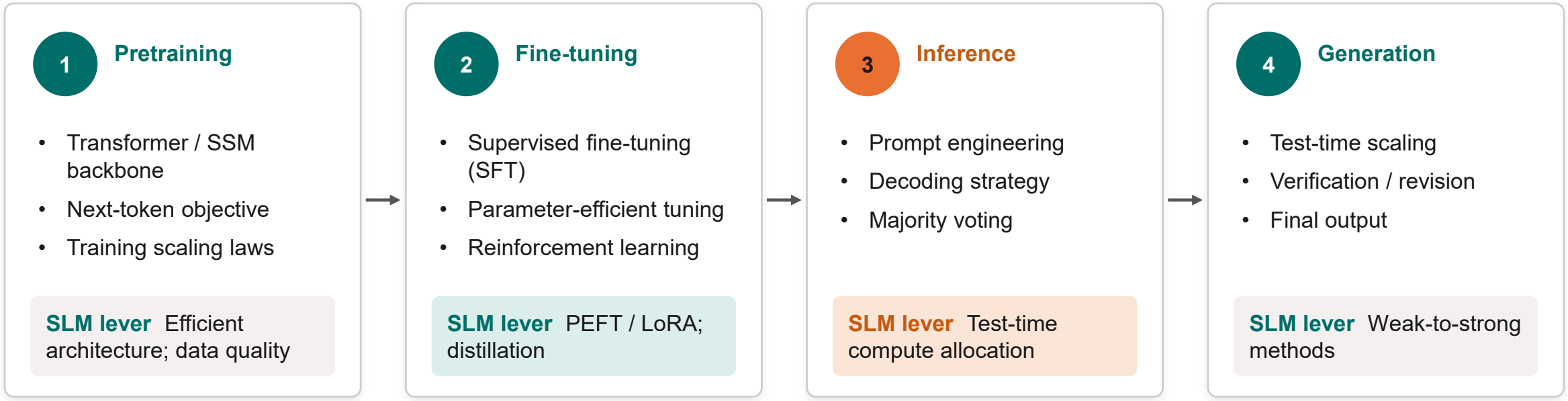
06

Trustworthiness

Robustness, safety, privacy, fairness

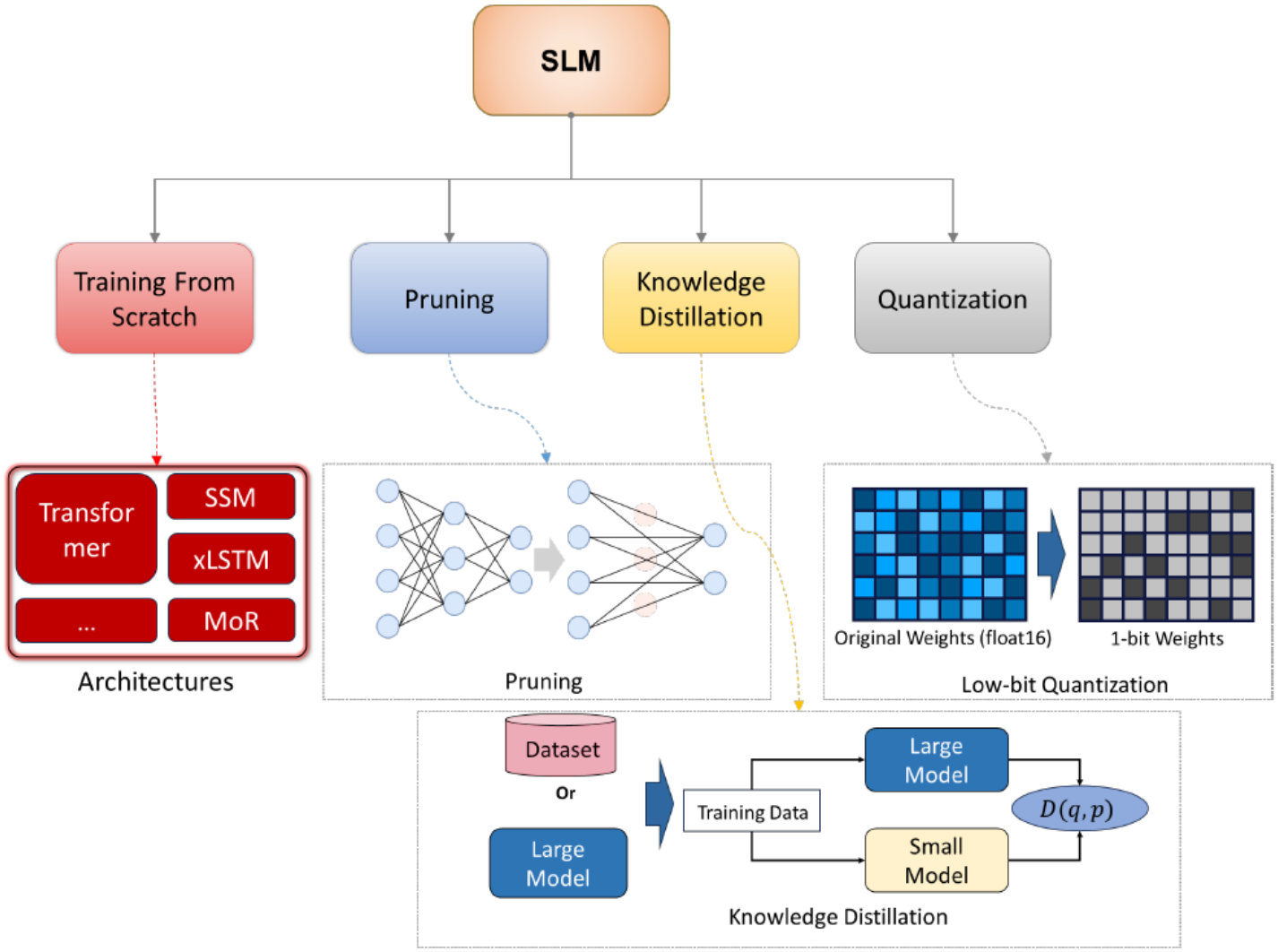
The Language Model Pipeline

A typical model moves from pretraining to inference. SLM optimization can enter at every stage — not only by shrinking the model.



The architecture is the same for large and small models; what differs for SLMs is the emphasis on efficiency, specialization, and inference-time strategies that make up for fewer parameters.

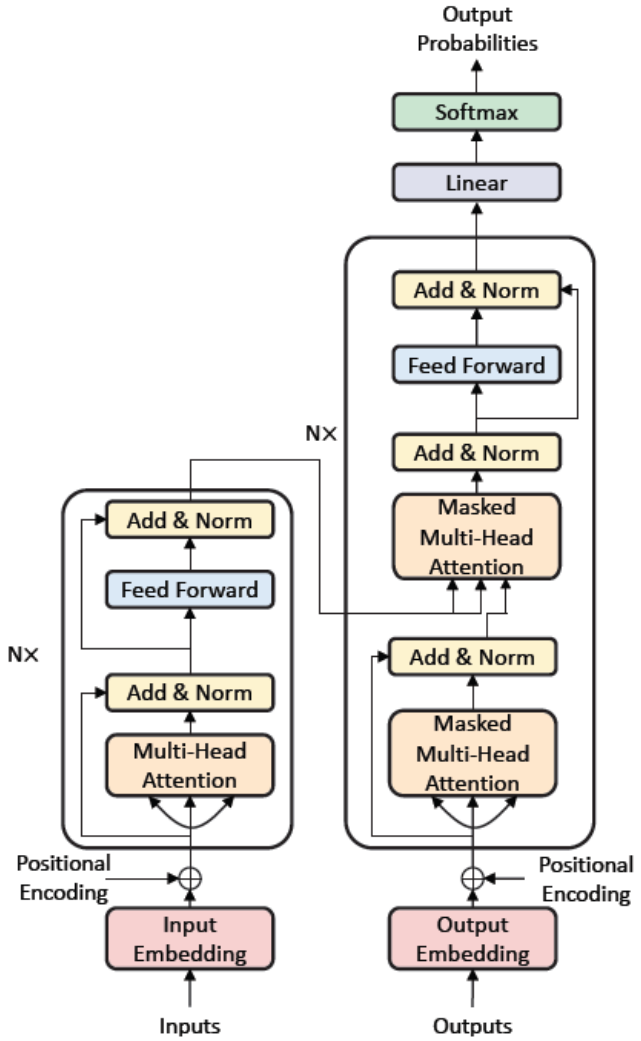
How to acquire small language models



Lecture Overview

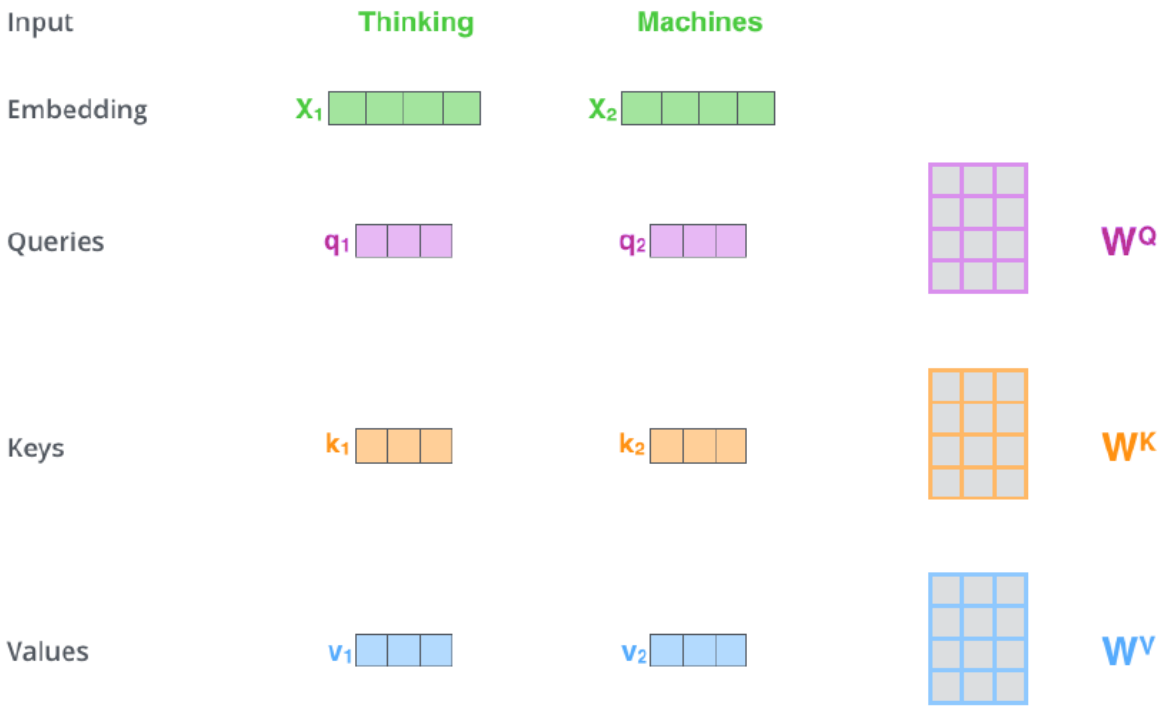
- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

Transformer



- Encoder: $p(x_i|\{x_j\}_{j\neq i})$
  - Position-wise feed-forward networks
  - Multi-head self-attention
  - Feedforward Neural Network
  - Residual connections and layer normalization
- Decoder: $p(x_i|x_{j<i})$
  - Position-wise feed-forward networks
  - Masked Multi-head self-attention
  - Feedforward Neural Network
  - Residual connections and layer normalization

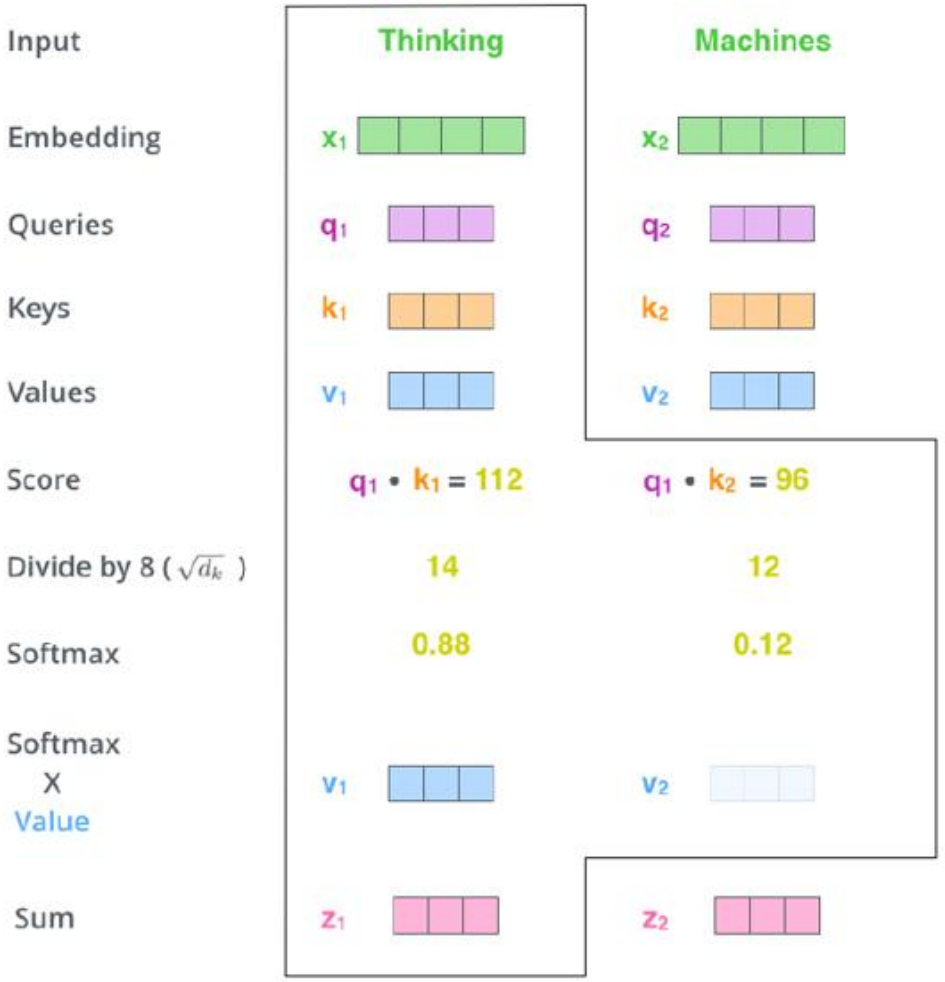
Transformer: self-attention



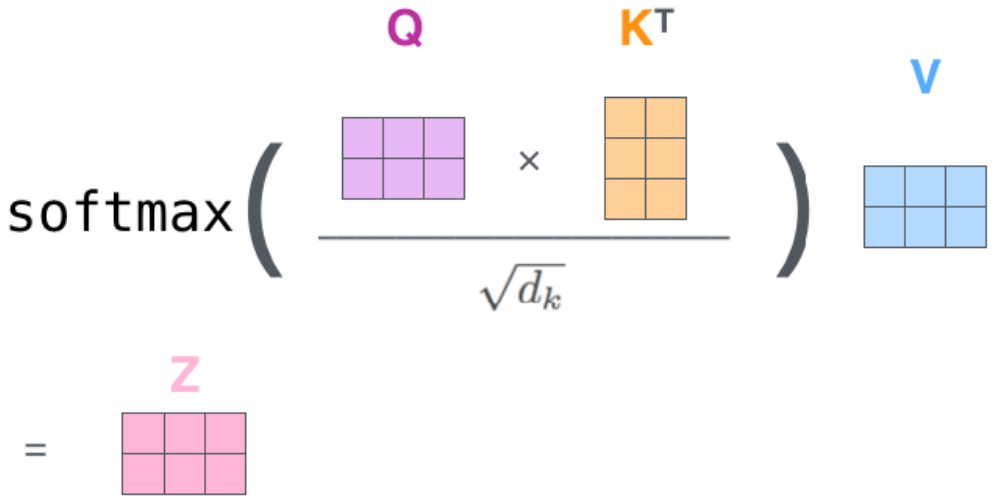
Multiplying x_1 by the W^Q weight matrix produces q_1, the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

[7] Figure credit (including the next several slides regarding self-attention): Jay Alammar. *The Illustrated Transformer*. Blog in 2018.

Transformer: self-attention



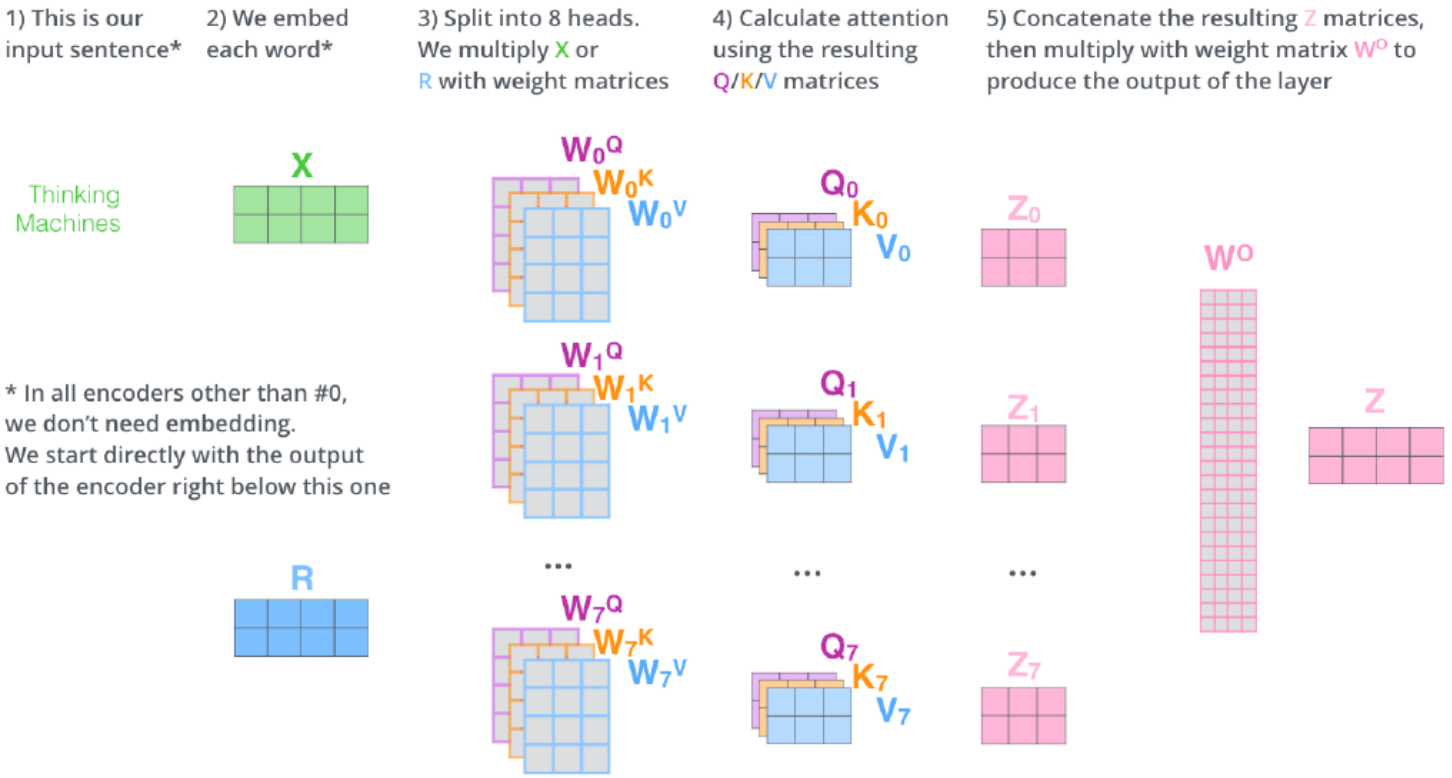
Transformer: self-attention



The self-attention calculation in matrix form, where d_k refers to the dimension of query/key vectors.

- Self-attention enables the model to weigh the importance of different words in an input sequence, allowing it to understand the context and capture dependencies between words.

Transformer: multi-head self-attention



Multi-Head Self-Attention adopts multiple attention "heads" in parallel and concatenates their representations, allowing the model to capture different types of linguistic patterns and dependencies, such as syntax, semantics, and positional relationships, simultaneously.

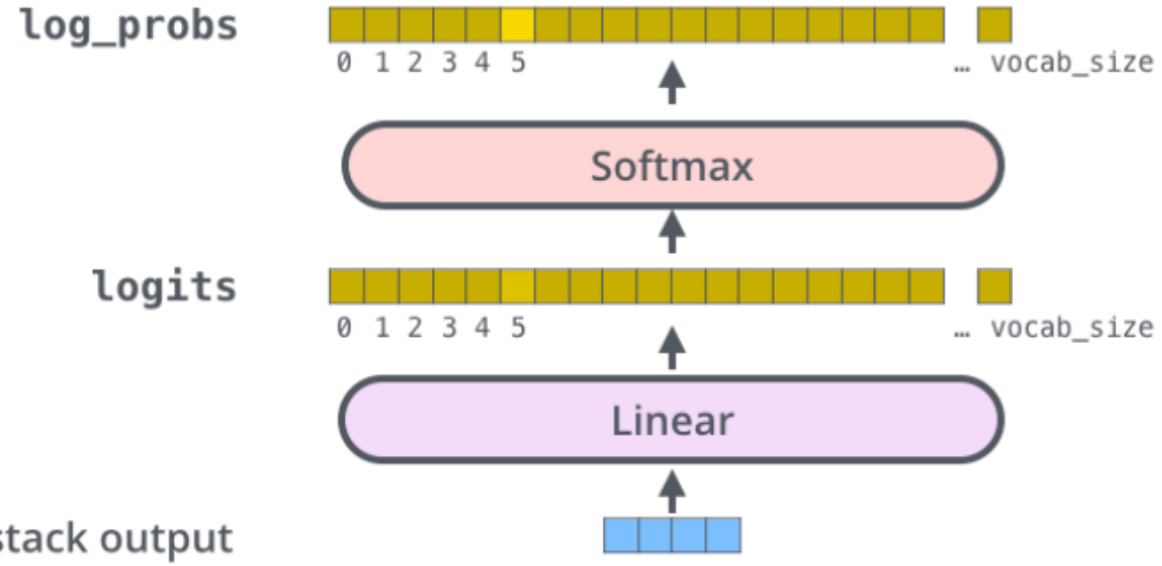
Transformer: softmax output

Which word in our vocabulary is associated with this index?

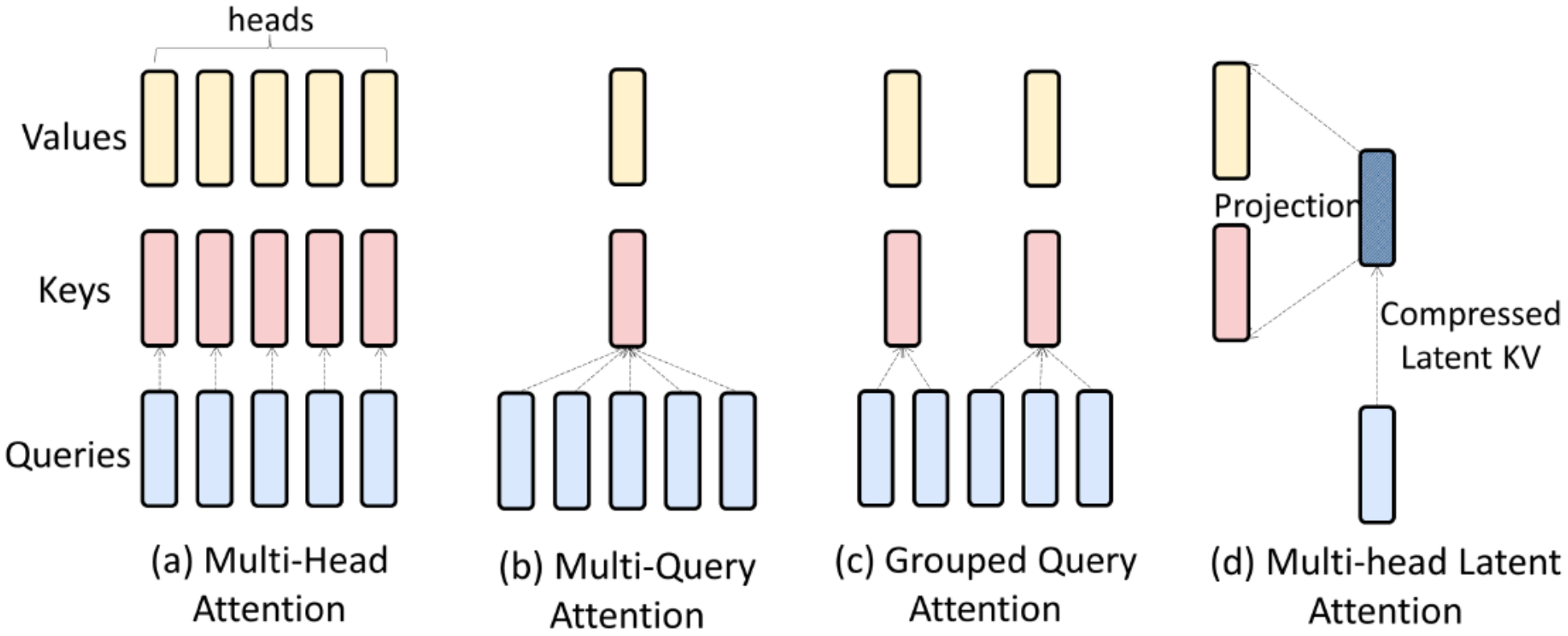
am

Get the index of the cell with the highest value (**argmax**)

5



Transformer: attention types



Lecture Overview

- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

Transformer Design Choices for SLMs

SLMs keep the Transformer skeleton but select each component for efficiency. Reported preferences across recent sub-billion and few-billion models:

Component	Common options	Typical SLM preference	Rationale
Attention	MHA · MQA · GQA · MLA	**Grouped-query (GQA)**	Balances representation power against KV-cache size
FFN activation	ReLU · GELU · Swish	**Swish / SwiGLU**	More expressive per parameter
Normalization	LayerNorm · RMSNorm	**RMSNorm**	Cheaper; no centering, stable enough
Parameter sharing	Embedding / layer / FFN	**Embedding + block sharing**	Reuses weights to cut parameter count
Shape	Wide-and-shallow · deep-and-thin	**Deep-and-thin**	Better accuracy at fixed budget on-device

Why Architecture Matters More at Small Scale

With only millions to a few billion parameters, an SLM has little redundant capacity to absorb inefficient design. Choices that are second-order for a 70B model become first-order for a 0.5B one.



Tight parameter budget

Every parameter must earn its place; wasted capacity directly lowers achievable quality.



Capacity allocation

Depth vs. width, attention sharing, and FFN size decide where limited parameters are spent.



Hardware alignment

Runtime speed on the target device can be more sensitive to architecture than to loss.



Compounding overhead

Cache size, activation memory, and I/O dominate; small structural savings compound on-device.

Design principle

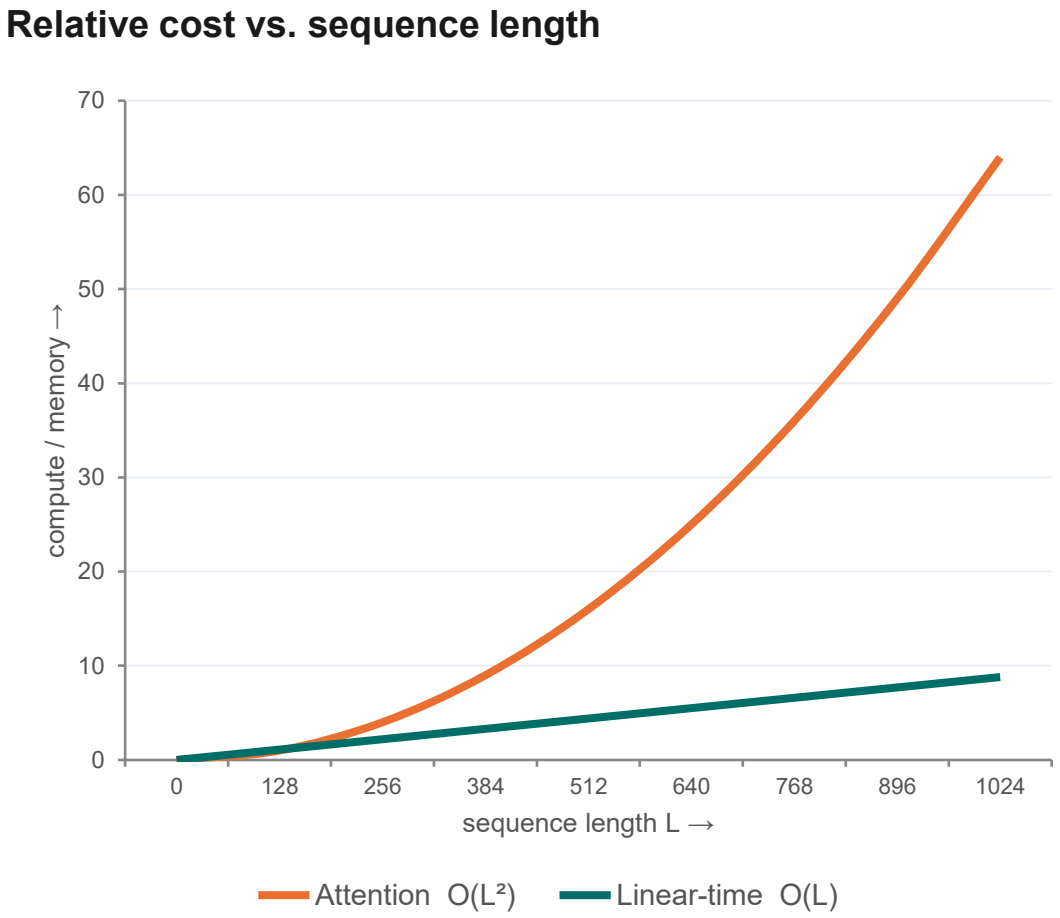
“Allocate scarce capacity where it yields the most capability for the target task and device.”

- Efficient attention shrinks the KV cache
- RMSNorm and SwiGLU add expressiveness cheaply
- Parameter sharing reuses weights across layers

Attention Efficiency and the Cost of Length

Self-attention compares every token with every other token. Compute and memory therefore grow quadratically with sequence length L — the dominant bottleneck for long-context, on-device use.

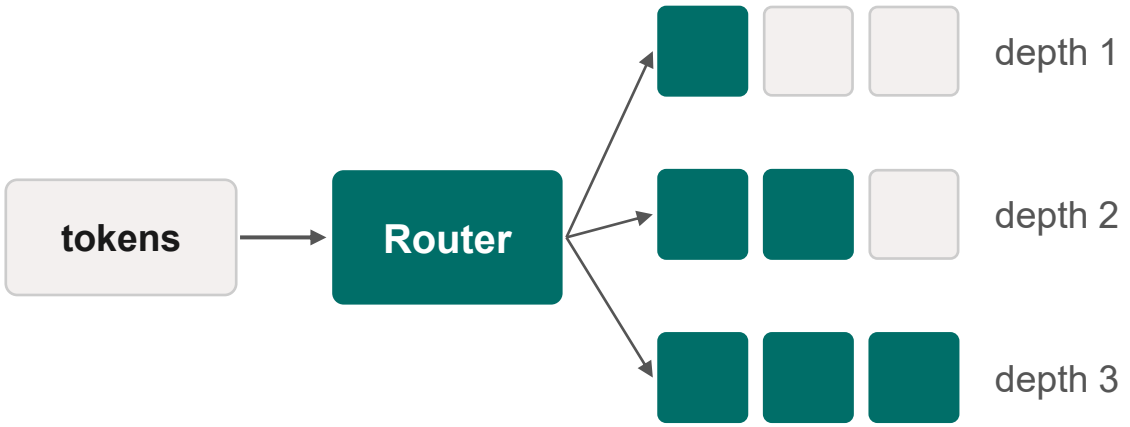
- **Training.** Full attention matrices are computed in parallel; all pairwise dependencies resolve at once — fast, but $O(L^2)$ memory.
- **Inference.** Each new token attends over the entire past, giving $O(L^2)$ total work and a KV cache that grows linearly with L.
- **Implication.** Long contexts stress memory first; this motivates sub-quadratic alternatives (SSMs, xLSTM, recursion).



Routing-Based Efficiency: Mixture-of-Recursions

Instead of adding parameters, MoR reuses a shared block and lets a router decide how many times each token passes through it — spending computation only where it is needed.

- **Recursive transformers.** Reuse the same layers repeatedly — weight sharing keeps the memory footprint low.
- **Router.** Assigns a dynamic, per-token recursion depth based on difficulty.
- **Efficiency gains.** A smaller KV cache plus depth-wise batching yields faster inference.



computed vs. skipped passes

Adaptive, token-level computation lets a small shared model behave like a deeper one only when the input demands it — a promising direction for compute-efficient SLMs.

Comparing Architectures for SLMs

Each family trades off training parallelism, inference cost, and memory differently. None is a universal winner — the right choice depends on task, context length, and hardware.

	Core mechanism	Training	Inference	Memory	SLM relevance
Transformer	Full self-attention	Highly parallel	$O(L^2)$, slower	KV cache grows with L	Default; efficient with GQA / sharing
Mamba (SSM)	Selective state space	Parallel scan	Linear-time	Constant-size state	Fast long-context; not SLM-tuned yet
xLSTM	Gated recurrent memory	Parallel (mLSTM)	Linear-time	Matrix / cell memory	Strong long-range; cross-modal
MoR	Routed recursion	Shared-weight blocks	Adaptive depth	Reduced KV cache	Compute only where needed

Takeaway: sub-quadratic models reduce inference and memory cost; Transformers retain the richest tooling and ecosystem support.

Lecture Overview

- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

On-Device SLM Deployment

Running a model on the device removes the network but imposes hard limits. Sub-billion designs target these constraints directly.



Memory

Keep the footprint small relative to device RAM

Power

Energy per token bounds battery life

Latency

Real-time response without a round trip

Offline

Must function with no connectivity

Privacy

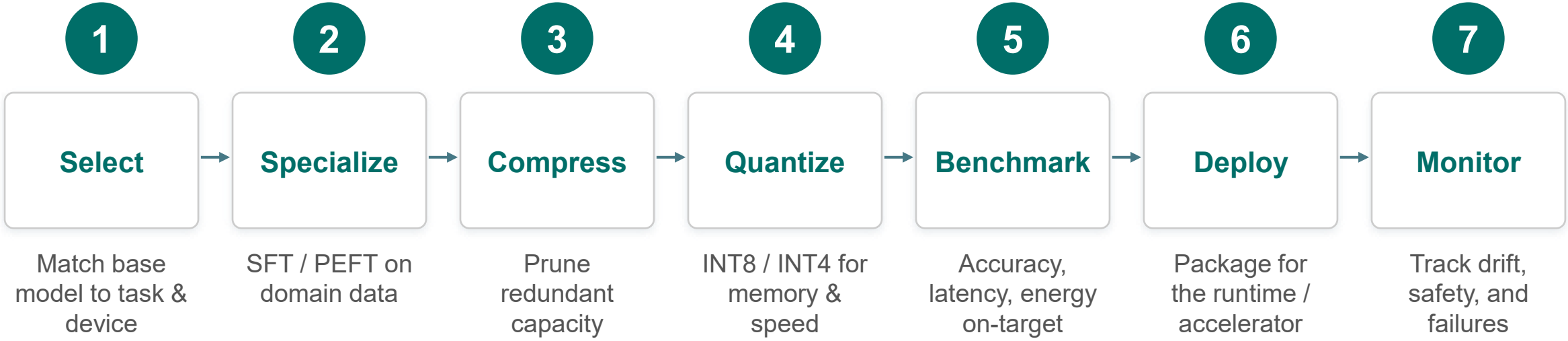
Sensitive data never leaves the device

Hardware

Adapt to the target SoC / accelerator

An SLM Deployment Pipeline

Getting from a candidate model to a reliable on-device system is an engineering workflow. Each stage carries one decisive consideration.



The loop is iterative: monitoring feeds back into re-specialization and re-benchmarking as data, tasks, and hardware evolve.

When Is an SLM the Right System Choice?

SLMs are not universally preferable. Weigh each dimension against the task; the right answer is often a hybrid system.

Dimension	Favor an SLM	Favor an LLM
Task scope	Narrow, well-defined	Broad, open-ended
Latency need	Real-time / on-device	Tolerant of round trips
Privacy need	High — keep data local	Lower / server-side OK
Memory budget	Constrained	Ample (cloud)
Cost sensitivity	High — minimize per-query	Lower priority
Connectivity	Intermittent / offline	Reliable network
General reasoning	Limited demand	Heavy demand

Lecture Overview

- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

Trustworthiness of SLMs

On-device deployment raises the stakes on trust. We organize SLM trustworthiness into five dimensions, each with distinct risks and mitigations.

1



Robustness

Resistance to adversarial and out-of-distribution inputs

2



Reliability

Truthful, well-grounded output; less hallucination and sycophancy

3



Safety

Resistance to jailbreaks and harmful generation

4



Privacy

Protection against data extraction and inference attacks

5



Fairness

Avoiding biased or stereotyped behavior

A recurring finding: model size is not a reliable predictor of trustworthiness — training and alignment choices matter more.

Jailbreak and Safety Risks

Jailbreaks craft prompts that bypass safety filters.

Key findings

- Model size is not a reliable predictor of jailbreak robustness.
- Training strategy and alignment matter more: SFT tends to be safer than DPO, which shows compliance drift.
- Knowledge distillation can increase jailbreak vulnerability.

Why SLMs are exposed

- Local IoT deployment lowers the attack threshold
- Often prioritize helpfulness over harmlessness
- Resource limits hinder effective guardrails

Toward inherent defenses

- Build safety into training / alignment, not just prompts (RLHF, adversarial training)
- Gradient- and logit-level analysis for detection

Safety is a function of design, not size.

Privacy Risks

As user-facing edge devices, SLMs can leak sensitive information, risking regulations such as GDPR and CCPA. PrivLM-Bench categorizes three major attack types.

Data extraction

Recover memorized training content by prompting the model with partial information.

Membership inference

Determine whether a specific example was part of the fine-tuning corpus.

Embedding inversion

Reconstruct private attributes of an input from its embedding vector alone.

Empirical finding: SLMs show limited privacy defense even under moderate attack conditions. As deployment moves to edge and user-facing settings, privacy-preserving capability becomes a first-class requirement.

Privacy and Fairness Mitigation

Both privacy and fairness are best addressed across the model lifecycle rather than as a single post-hoc fix.

	Data	Training	Model	Inference
Privacy	Local, on-device processing	Differential privacy (noise in training)	Quantization-aware DP (open challenge)	DP prompt tuning (noisy soft prompts)
Fairness	Data augmentation (balance representation)	Auxiliary fairness objective / regularizer	Model editing (correct biased behavior)	Prompt-level filtering at inference

Interactions matter: e.g., differential privacy can be fragile under quantization, and quantization can amplify bias on-device.

Lecture Overview

- Language models: the basics
- Why small language models?
- What is a small language model?
- Just enough Transformer
- Architectures for SLMs
- Deploying SLMs
- Trustworthiness
- Outlook

Open Research Directions

Open problems cluster into six themes spanning architecture, data, systems, and trust.

Efficient architectures

SLM-specific designs that keep training fast without slow inference (e.g., Mamba, xLSTM adapted for small scale).

LLM-to-SLM data

High-quality synthetic data generation that avoids distribution mismatch when teaching SLMs from LLMs.

Personalization & proxy

On-device LoRA personalization; SLMs as cost-effective proxies for optimizing and steering LLMs.

Cloud–edge & systems

Deeper cloud–edge synergy and multi-agent SLM collaboration as scalable alternatives to single LLMs.

Trustworthiness evaluation

Unified, standardized benchmarks for SLM robustness, safety, privacy, and fairness.

Retrieval for SLMs

RAG tailored to SLMs — lightweight, graph-structured retrieval to support multi-step reasoning.

Small Models, Deliberate Design

1. **“Small” is relative.** Task requirements and resource budgets define an SLM — not a fixed parameter count.
2. **Architecture is decisive.** Under tight budgets, efficient attention, normalization, and routing choices drive capability.
3. **Specialization offsets scale.** Fine-tuning lets small models match or exceed larger ones on focused tasks.
4. **Deployment reshapes risk.** On-device advantages come with new safety, privacy, and fairness requirements.
5. **SLMs and LLMs are complementary.** Increasingly they operate together — proxies, cloud–edge, and multi-agent systems.

SLMs are not a smaller footnote to LLMs but a distinct, deliberate design space — where capability is earned through efficiency, specialization, and trustworthy engineering.